Practical- 04

Title of the Assignment: Local Beam Search for the Travelling Salesman Problem

Problem statement: Implement local beam search to obtain a short tour for the Travelling Salesman Problem (TSP), in which every city is visited exactly once and the salesman returns to the starting city.

Objective:

To understand local search and population-based heuristic search techniques.

To implement local beam search using multiple candidate tours maintained at every iteration.

To evaluate a TSP tour using total distance and compare the quality of the best solution across iterations.

Theory:

Travelling Salesman Problem:

The Travelling Salesman Problem consists of a set of cities and the distance between each pair of cities. The objective is to find a minimum-cost Hamiltonian cycle: a tour that starts at one city, visits every other city exactly once and returns to the starting city. The problem is computationally difficult for large inputs, so heuristic search methods are often used to obtain good solutions in reasonable time.

Local beam search:

Local beam search maintains k candidate states instead of a single current state. At each iteration, successors are generated from all k candidates. The best k successors, according to the evaluation function, become the candidates for the next iteration. In this assignment, each state is a complete TSP tour and its cost is the total length of the closed tour. Lower cost represents a better state.

A tour can be represented as a permutation of city indices. Neighbouring tours may be generated using swap, insertion or 2-opt operations. A 2-opt move reverses a segment of the tour and is especially useful because it can remove crossing edges. To avoid losing the best solution found so far, the implementation should use a global best-tour record even if a later beam contains only inferior tours.

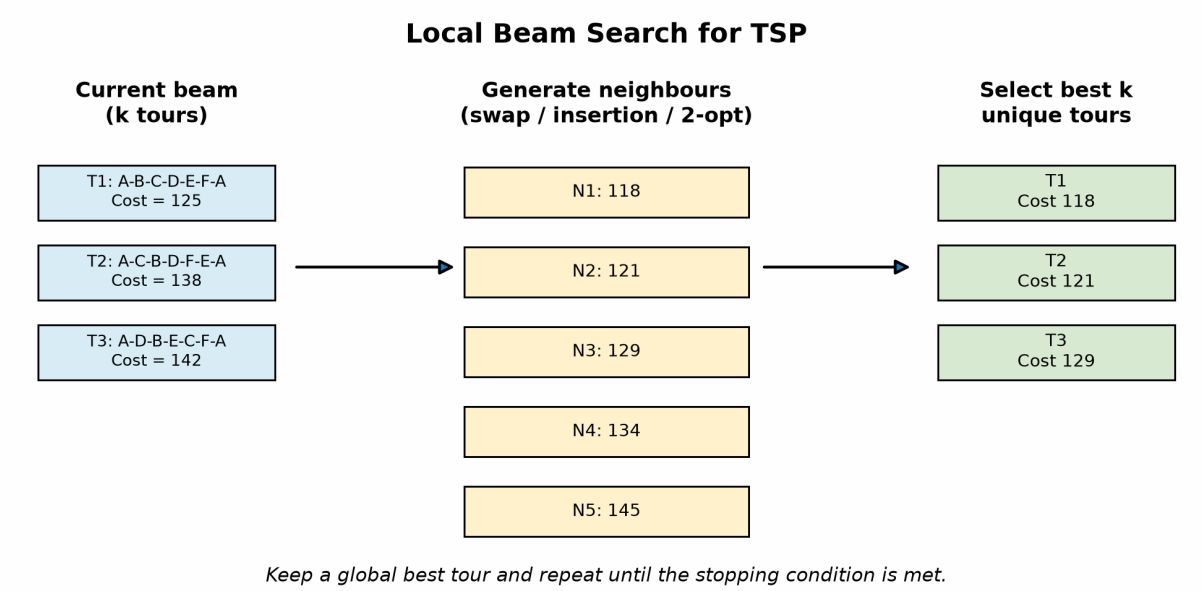


Figure 1: Local beam search keeps the best k unique successor tours

Problem formulation:

State: A complete permutation of all cities, with an implicit edge from the last city back to the first city.

Initial states: k different tours generated randomly or by combining a fixed starting city with shuffled remaining cities.

Successor function: Generate neighbouring tours using swap, insertion or 2-opt transformations.

Evaluation function: tour_cost = distance(last, first) + Σ distance(tour[i], tour[i+1]).

Selection rule: Keep the k lowest-cost unique successor tours for the next beam.

Algorithm:

Step 1: Start.

Step 2: Read the city coordinates or the distance matrix, beam width k and maximum number of iterations.

Step 3: Generate k initial tours and calculate the cost of every tour, including the return edge to the starting city.

Step 4: Store the lowest-cost tour as the current global best tour.

Step 5: Generate neighbouring tours for every tour in the current beam using the chosen neighbourhood operation.

Step 6: Remove duplicate tours and calculate the cost of every remaining successor.

Step 7: Sort the successors in ascending order of tour cost.

Step 8: Select the first k successors, or all remaining successors if fewer than k are available, to form the next beam.

Step 9: Update the global best tour if the best tour in the new beam has lower cost.

Step 10: Stop if the maximum number of iterations is reached or if no improvement occurs for a specified number of iterations.

Step 11: Otherwise, repeat Steps 5–10.

Step 12: Display the best tour, its total distance and the iteration at which it was found.

Step 13: Stop.

Suggested experiment:

Use the following six-city distance matrix, where the diagonal entries are zero and the matrix is symmetric. City labels may be A, B, C, D, E and F.

	A	B	C	D	E	F
A	0	10	15	20	18	12
B	10	0	35	25	17	28
C	15	35	0	30	28	20
D	20	25	30	0	22	16
E	18	17	28	22	0	14
F	12	28	20	16	14	0

Use a beam width such as k = 4, generate at least 10 neighbours per current tour where possible, and run the algorithm for a fixed number of iterations. Because local beam search is heuristic and may depend on the initial beam, repeat the experiment with more than one random seed and report the best and average tour cost.

Expected output:

The output should display the beam width, iteration number, best cost in the current beam, global best cost and the final tour. The final report should include the distance matrix or input coordinates, neighbourhood operation, stopping condition and a brief comparison of results for different beam widths.

Conclusion:

Local beam search was implemented for the Travelling Salesman Problem by maintaining multiple candidate tours and repeatedly selecting the best successors. The experiment demonstrates the effect of beam width, neighbourhood design, duplicate handling and random initialization on the quality of a heuristic solution.

Oral questions:

1. What is the Travelling Salesman Problem?
2. Why is TSP called a combinatorial optimization problem?
3. What is local search?
4. How does local beam search differ from hill climbing?
5. What is the role of beam width k?
6. Why should duplicate tours be removed?
7. What is a 2-opt move and why is it useful for TSP?
8. Why is the return distance from the last city to the first city included?
9. What is the difference between the current beam best and the global best tour?
10. How can random restarts or different beam widths improve the result?

References:

[1] Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach, section on local search algorithms.

[2] David S. Johnson and L. A. McGeoch, The Traveling Salesman Problem: A Case Study in Local Optimization.